

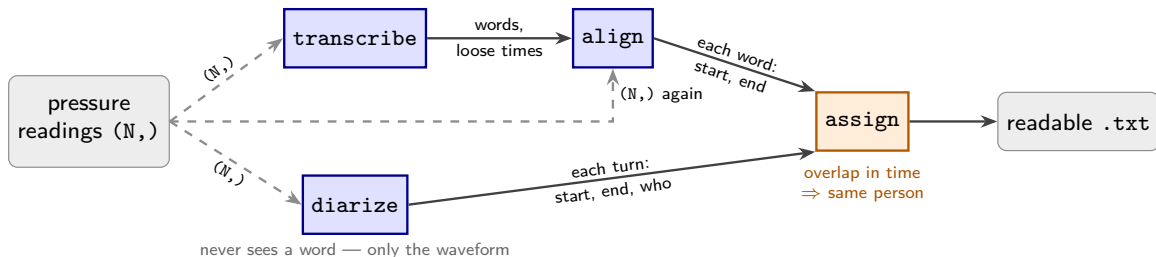
How Audio Becomes a Transcript

What `run_whisperx.py` accomplishes, step by step

PSYCH-ASR — Stage 1

The whole file is five calls

Call	Question it answers	In	Out
load_audio	What are the numbers?	file	(N,)
transcribe	What words were said?	(N,)	text, per chunk
align	<i>When</i> was each word said?	text + (N,)	word times
diarize + assign	<i>Who</i> said it?	(N,)	speaker labels
render	Can a human read it?	transcript	a .txt play script



Dashed = the *same* array, read from scratch three times — the models share no internals. Solid = text moving forward.

Step 1 — load_audio: sound becomes numbers

IN: audio file (.wav) $\longrightarrow$ **OUT:** float32 array, (N,)

A microphone measures **air pressure**. 16,000 times per second we write down the current pressure as a number between -1 and 1 . That flat list of N numbers *is* the audio — everything downstream reads it.

$$N = \text{seconds} \times 16000 \quad \implies \quad 50\text{-minute session} = 48,000,000$$

The one trick to remember: time converts to position by multiplying by 16000. A word starting at 4.5 seconds begins at index 72,000. Every timestamp in this pipeline is ultimately a statement about where you are in this array.

Step 2 — transcribe, part 1: audio becomes a picture

IN: (N,) $\longrightarrow$ **OUT:** one picture per 30-second chunk

One call doing five jobs. The first three just get the audio into a shape the model can eat:

- **Find the speech.** A small model marks speech vs. silence. It transcribes nothing — it only draws boundaries.
- **Pack into chunks.** Whisper only ever swallows 30 seconds at a time — that is the size of its mouth and it never changes. So speech stretches get taped end-to-end until the next one would push past 30 seconds; then a new chunk starts. **Nothing about the conversation decides where that cut lands** — only the 30-second ceiling does. Chunks do not depend on each other, so 16 can go through the GPU at once. That is `batch_size`.
- **Turn each chunk into a picture.** A spectrogram: time along one axis, pitch along the other, brightness = how much of that pitch is present.

Step 2 — transcribe, part 2: picture becomes words

IN: one picture per chunk → **OUT:** segments of text

The other two jobs are the two neural halves of Whisper:

- **The encoder reads the picture.** A 20 ms sliver of sound means nothing by itself — the same buzz sits inside “said” and “sat”, and only the sound around it settles which. So every sliver gets to look at every other sliver in the 30 seconds and rewrite itself with that context. That looking-around is **self-attention**, and a sliver 25 seconds away is just as easy to reach as the one next door. Do it enough times and each sliver describes its moment *in the context of the whole chunk*.
- **The decoder writes the words**, one token at a time, like a chatbot: predict, append, predict again — reading the encoder’s output as it goes, because that is the only channel through which the audio reaches it.

What transcribe hands back

start	end	text	avg_logprob
0.0	26.0	"so how has this week been for you ..."	-0.09
27.0	55.0	"I guess it started okay but then ..."	-0.14

Read those timestamps carefully. They are *chunk boundaries* — artifacts of the 30-second packing. They are not sentence boundaries, and they were never measured from the speech.

Not one individual word has a timestamp yet. Each segment is one undifferentiated string. That hole is the entire reason the next step exists.

(avg_logprob is the model's confidence, and Stage 2's quality triage runs on it. Above -0.5 is a confident decode; below -1.0 is where hallucination starts.)

Step 3 — align: this is karaoke

IN: segments + (N,) $\longrightarrow$ **OUT:** segments, each with a "words" list

Karaoke never guesses the lyrics. It was handed them. All it decides is which word lights up at which moment.

That is this step. `transcribe` gave us the words but not their times, so a smaller model (`wav2vec2`) listens to the raw audio again and walks the known words against it in order.

Because the words are handed to it, it cannot make any up — it cannot skip, add, or reorder. It only decides where each word sits on the clock.

word	start	end	score
so	0.31	0.44	0.91
how	0.46	0.63	0.88
has	0.65	0.79	0.94

Now we know **what** was said and **when**. Still no idea **who** said it. (Side effect: segments come back re-split at sentence boundaries, so the segment count goes *up*.)

Step 4 — diarize: whose voice is this?

Diarization does not use the Whisper *model*.

The class lives in `whisperx.diarize`, but that file is a thin wrapper — every model it runs comes from `pyannote`. It never sees the transcript or the words; it starts over from the raw (N,) array.

Whisper asked what sounds were made. This asks who made them — and you can tell two strangers apart in a language you do not speak. It takes three moves.

Move one: find the talking. Mark every stretch where somebody is speaking — and every stretch where **two people are speaking at once**.

Spotting an overlap needs no idea *who* is talking. Two tangled voices simply sound different from one, the way you can hear that a room is crowded without recognising a single person in it.

Overlapping stretches sit out the next move, because a smear of two voices describes neither of them. **Nothing is deleted** — Whisper transcribed those words long ago and they stay in the transcript.

Move two — turn each voice into numbers

IN: one stretch of speech → **OUT:** a list of 256 numbers

A second model listens to one clean stretch and turns it into a list of **256 numbers**. Always 256 — whether the stretch ran one second or twenty.

Why numbers? Because two voices cannot be compared directly, but two lists of numbers can.

The model was trained on thousands of speakers with one rule: two clips of the *same* person must come out with similar numbers, two clips of *different* people far apart. It was never taught a single word.

So what survives in those 256 numbers is the **voice** — pitch, resonance, breathiness — and nothing about what was said. A muttered “mm-hm” and a two-minute story from the same person still come out looking alike.

Move three — cluster, and what comes back

Cluster. Sort the 256-number fingerprints into piles by how close together they sit. Each pile is one speaker, and every stretch in that pile gets its label:

start	end	speaker
0.3	3.9	SPEAKER_00
4.1	9.5	SPEAKER_01
9.8	14.8	SPEAKER_00

Times and speakers. **No words anywhere.** This table has never seen the transcript.

Why we pass `--num-speakers 2`. Left alone, clustering guesses the number of groups and errs both ways: it splits one person in two when their voice shifts (calm vs. distressed patient), and merges two people whose voices are similar. Every pilot session has exactly two people in it, so we say so and remove both failure modes.

Labels are arbitrary — SPEAKER_00 is not the therapist. Mapping labels to roles is Stage 2's first job.

Step 5 — assign: stack the two lists

We now hold two lists built in total ignorance of each other, sharing exactly one thing: **a clock**. `align` says *which word* happened when. `diarize` says *who was talking* when. Lay one on top of the other and read the answer off.

For each word, look at whose stretch it lands inside; if it straddles a handover, whoever covers more of it wins:

turn	span	overlap with “okay” (3.0–3.4)	
SPEAKER_00	0.0–3.1	0.1 s	
SPEAKER_01	3.1–8.0	0.3 s	← winner

This is why alignment had to run first. Without it we are not placing a word, we are placing a 26-second chunk — and half a minute of dialogue gets stamped onto whoever dominated it.

Step 6 — render: nobody wants to read JSON

IN: transcript with speakers → **OUT:** <stem>.transcript.txt

The check this whole project hinges on is a **human** listening to the audio with the transcript open. That person needs a play script, not a data structure. One more pass — pure formatting, no model, CPU, sub-second — groups consecutive segments sharing a speaker into one block and prints it:

```
[00:00] SPEAKER_00
    So how was the week?
[00:04] SPEAKER_01
    It was rough.  I did not leave the house Tuesday.
[00:15] UNKNOWN
    Mm-hm.
```

Grouping matters because alignment re-split everything at sentence boundaries — ungrouped, a minute of speech reads as a list of sentences, not a conversation. UNKNOWN is printed rather than hidden: a cluster of them *is* the finding.

End to end

Step	In	Out
load_audio	audio file	(N,) float32
transcribe	(N,)	segments: text + chunk bounds
align	segments + (N,)	segments + per-word start/end/score
DiarizationPipeline	(N,)	turns table: start, end, speaker
assign_word_speakers	turns + transcript	speaker on words/segments
render_transcript	labelled transcript	grouped, timestamped .txt

Three models, one waveform read three times, joined at the end on nothing but timestamps.

It all lands in word_segments

Everything above collapses into one key of `data/stage1/<stem>.diarized.json`: a flat list, **one entry per word**, where each entry answers every question the pipeline was built to ask.

field	answers	came from	example
word	what was said	transcribe	"okay"
start, end	when	align	3.02, 3.41
score	how sure we are of the timing	align	0.88
speaker	who said it	diarize + assign	SPEAKER_01

That is the whole deliverable. Every Stage 3 feature — talk-time ratio, turn counts, speech rate, silence, interruptions, who asked what — is arithmetic over this one list. (`segments` is the same content re-grouped into sentences for reading; `language` is "en".)

First pilot session: **7,298 words**, of which 7,293 carry all five fields and 5 have no speaker — those five landed in moments diarization had written off as silence, and there is no fallback guess, so the field is missing. Whether the other 7,293 labels are *correct* is the human check nothing downstream should be built before.